Track 1: Quantum Chemistry

Ground-state energies and molecular structure on quantum hardware

Qiskit Fall Fest Hackathon · Oct 9, 2026 · IISER Bhopal (EECS Club and Physics Club)

Read this first. You have one week, a laptop, and AI assistants. The ten problem statements are suggestions, not a menu you must order from. You may pick one, extend one, combine several, or invent something more ambitious. Original, advanced and cross-track projects are welcome and will be judged on the same criteria.

What we reward. A project that works, that you can explain line by line, and that is honestly compared against a classical alternative. Using AI to write code is allowed and encouraged; being unable to explain or verify that code is not.

How to read the problem statements

Every problem has the same parts: a short background, the problem itself, the core tasks (what a complete submission does), stretch goals (what makes it excellent), and a hint on tools. Treat the core tasks as the minimum, not the ceiling.

Simulators first, hardware as a bonus. Everything here can be done on a simulator (Qiskit Aer or the statevector tools). Running a small version on real IBM hardware, with a clear comparison against the simulator, earns extra credit in any track.

This track contains: **Problem 1, Problem 2**.

Problem 1: The hydrogen molecule with VQE

Background

Hydrogen (H_2) is the smallest molecule where electrons genuinely interact, and the standard first test for quantum chemistry on quantum computers. Its ground-state energy depends on the distance between the two atoms, and tracing that dependence gives the bond curve that tells chemists how strong the bond is and where it sits.

Problem

Compute the ground-state energy of H_2 across a range of bond lengths using the Variational Quantum Eigensolver, and decide how trustworthy your answer is.

Core tasks (the minimum for a complete submission)

1. Build the molecular Hamiltonian for H_2 (minimal basis, for example STO-3G) and map it to qubits. Reduce it to as few qubits as you can using symmetries, and say how.
2. Implement VQE with at least two different ansatz circuits (for example a chemistry-inspired one and a hardware-efficient one) and a classical optimiser of your choice.
3. Plot energy against bond length from about 0.3 to 3 Å, and mark the equilibrium bond length and the dissociation limit.
4. Compare against exact diagonalisation of the same Hamiltonian. Report the energy error in Hartree and say whether it is below chemical accuracy (about 1.6 milliHartree).

Stretch goals (what makes it excellent)

- Add a noise model and show how error grows, then apply an error mitigation technique and measure the improvement.
- Run one bond length on real IBM hardware and compare it with the simulator.
- Compute an excited state (for example with a variational method such as VQD) or the molecular dipole.
- Report how the number of circuit evaluations and the optimiser choice affect convergence.

Hints on tools

Qiskit Nature with PySCF builds the Hamiltonian; the Estimator primitive evaluates energies. A good submission explains why the ansatz is or is not expressive enough, not only that the numbers match.

Problem 2: Cyclohexane with sample-based quantum diagonalisation (SQD)

Background

Cyclohexane (C_6H_{12}) is a ring that can fold into several shapes (conformers), such as the chair, the boat and the twist-boat. The chair is the most stable, and the energy gaps between shapes decide how the molecule behaves. A molecule this size is far beyond what plain VQE can handle on any current device. Sample-based quantum diagonalisation takes a different route: a quantum computer samples important electron configurations, and a classical computer then solves the Hamiltonian restricted to those configurations.

Problem

Estimate the relative energies of cyclohexane conformers using SQD, and measure how well the sampled subspace captures the true ground state.

Core tasks (the minimum for a complete submission)

1. Choose a manageable active space (a handful of the most relevant orbitals and electrons) and justify the choice. Generate geometries for the chair and at least one other conformer.
2. Prepare a sampling circuit (for example a local unitary cluster Jastrow ansatz), sample bit strings, and run the SQD workflow, including the configuration recovery step that repairs samples corrupted by noise.
3. Report the energy of each conformer and the energy differences between them, in kcal/mol.
4. Compare against a classical reference (such as a CASCI calculation in PySCF, or a larger classical method) and report the error.

Stretch goals (what makes it excellent)

- Study convergence: plot the energy against subspace size and against the number of samples.
- Compare samples from an ideal simulator with samples from a noisy backend or real hardware.
- Identify which conformer ordering is reproduced and which is not, and explain why.
- Explore larger active spaces and report where the classical step becomes the bottleneck.

Hints on tools

IBM provides the open-source qiskit-addon-sqd package and tutorials; ffsim simulates the fermionic sampling circuits. Remember that you are expected to verify the literature ordering of the conformers rather than assume it. The honest question to answer is: how much of the accuracy comes from the quantum sampling and how much from the classical diagonalisation?

Go beyond: combine tracks or invent your own

The most impressive entries will not stop at one box. None of these is required, and you are free to propose something entirely different.

Ideas that involve Track 1

Idea	Tracks combined	What makes it interesting
Logical qubits inside an optimiser	Steane code and QAOA or VQE	Does error protection improve the answer on a noisy simulator, or does the overhead cancel the gain?
Barren-plateau-aware chemistry	VQE and QNN mitigation	Apply identity-block initialisation or layer-wise training to a larger VQE ansatz.

Open invitation. If you have your own idea, such as a quantum game, an algorithm tutor, a benchmark of a quantum algorithm from our workshop (Grover, phase estimation, the QFT), or a tool that makes any of the above easier for the next student, propose it. Write three sentences on the problem, the method, and how you will know it worked, and check with the organisers early in the week.

How to scope a combined project. Get the smallest version of each part working separately first, then connect them. A small, honest, working combination beats an ambitious one that does not run.

What every team submits

Whichever problem you choose (or invent), your submission has the same five parts:

- A code repository or notebook that runs from a clean start, with versions of the packages listed.
- A write-up PDF (two to four pages) covering the problem, method, results and an honest limits section.
- At least one figure that shows the main result.
- A short AI-use note: which tools you used, one thing the AI got wrong, and how you caught it.
- A five-minute demo followed by questions; every team member must be able to take questions.

Bonus. Bonus points are available for a hardware run on a real IBM backend, with a clear comparison against the simulator.

Full submission rules are in *docs/guidelines_and_submission_rules.pdf*.

How you are judged

The same scoring applies to every problem and to original projects alike.

Criterion	Points	What judges look for
Correctness and verification	30	Results checked against an exact or classical reference, tests, sensible error bars
Understanding and insight	25	Clear explanation of why the method works or fails, shown in the write-up and in the live questions
Honest comparison and limits	20	A fair classical baseline, stated assumptions, no unsupported claims of quantum advantage
Ambition and originality	15	Stretch goals, combined tracks, or a new idea beyond the suggested statements
Clarity of demo and code	10	Readable, reproducible code and a clear five-minute story
Total	**100**	Bonus points are available on top for a real IBM hardware run (see Bonus).

On using AI. AI assistants are welcome at every step. The AI-use note and the live questions exist because the person submitting is responsible for the result: you should be able to explain each circuit, formula and plot, and show how you verified it.